Podstawy baz danych

grupa: 13

dzień i godz zajęć: środa 13:15

nr zespołu: 5

Autorzy: Karolina Nitsch, Witold Nieć

https://github.com/kanitsch/bazy_danych

1. Wymagania i funkcje systemu

Projektowany system bazodanowy ma posłużyć firmie oferującej różnego rodzaju kursy i szkolenia w modelu hybrydowym. Usługi dzielą się na:

- Webinary
- Kursy
- Studia
- Pojedyncze spotkania studyjne

Składają się z następujących jednostek szkoleniowych:

- spotkania online asynchroniczne (kursy, studia)
- spotkania online synchroniczne (webinary, kursy, studia)
- spotkania stacjonarne (kursy, studia)
- praktyki (studia)
- egzaminy (studia)

W systemie wyróżniamy następujące role:

- Użytkownik niezalogowany
- Klient
- Nauczyciel
- Administrator
- Dyrektor
- Księgowy

Użytkownicy mogą korzystać z różnych funkcji, w zależności od ich ról w systemie. Role mogą być dziedziczone.

Funkcje

Funkcje systemu

- Zarządzanie dostepami do obszarów funkcjonalnych w oparciu o role w systemie

- Przyznawanie i odbieranie dostępu do jednostek szkoleniowych na podstawie wykupionych usług i terminów ważności.
- Zarzadzanie dostępnością usług na podstawie limitów miejsc
- Zarządzanie zaliczeniami jednostek szkoleniowych i produktów według zasad:
    - Studia - 80% frekwencji
    - Praktyki - 100% frekwencji
    - Kursy - zaliczenie 80% modułów
- Rejestrowanie i odnotowywanie obecności wszystkich uczestników spotkań online (z podziałem na role)
- Generowanie linków do płatności
- Rejestrownie płatności

Użytkownik niezalogowany

- Przegladanie, wyszukiwanie dostępnych produktów (z informacją o ich termianch, dostepnosci cenach):
    - Webinarów,
    - Kursów (wraz z programem),
    - Studiów (wraz sylabusem)
    - Zajeć studyjnych - dostępnych bez konieczności uczestnictwa w całych studiach
- Dostęp do ogólnych informacji na temat zasad funkcjonowania szkoły, regulaminów, formularzy komunikacyjnych

Klient (użytkownik zalogowany)

Dziedziczy funkcje Użytkownika niezalogowanego.

Ogólne

- możliwość zgłaszania problemów technicznych do administratora

Zakupy i zarządzanie dostępnymi szkoleniami

- Wyświetlanie liczby zapisanych osób, dostepnych tłumaczeniach i limicie miejsc dla danego produktu
- Dokonywanie zapisów poprzez koszyk zakupowy
- Generowanie żądania linków do płatności
    - płatność pełna za webinar - link ważny do momentu rozpoczecia webinaru
    - płatność pełna za zajecia studyjne - link ważny do 3 dni przed rozpoczeciem kursu
    - płatość pełna za kurs - link ważny do 3 dni przed rozpoczeciem kursu
    - płatność wpisowego za studia - link ważny w dniu zapisu
    - płatność za zjazd w ramach studiów - link ważny do 3 dni przed rozpoczeciem zjazdu
    - dla każdej płatnosci istnieje możliwość zawnioskowania o płatność odroczona (wymaga akceptacji Dyrektora)
- Integracja z operatorem płatności
- Wyświetlanie aktualnej listy zamówionych usług szkoleniowych

- Sprawdzanie kolizji

Webinary

- Wyswietlenie dostępnych webinarów z informacją o terminach
- Odtwarzanie darmowych webinarów
- Odtwarzanie płatnych webinarów z wykupionym dostępem i w terminie dostępności

Kursy

- Wyświetlenie wykupionych kursów ze statusem zaawansowania
- Wyświetlenie zawartości kursów z informacją o terminach
- Uruchamianie spotkan online asynchronicznych
- Dołączanie do spotkań online synchronicznych
- Sprawdzanie statusu obecności/zaliczenia
- Oglądanie nagrań ze spotkań online

Studia

- Wyświetlenie wykupionych studiów ze statusem zaawansowania
- Wyświetlenie sylabusu studiów z informacją o terminach
- Uruchamianie spotkań online asynchronicznych
- Dołączanie do spotkań online synchronicznych
- Sprawdzanie statusu obecności/zaliczenia
- Oglądanie nagrań ze spotkań online
- Odrabianie nieobecności

Nauczyciel

Ogólne

- Wyświetlanie kalendarza/planu zajęć
- Dołączanie do spotkań online (z odpowiednimi uprawnieniami zarządzania spotkaniem)

Spotkania online asynchroniczne

- Nagrywanie spotkania i udostepnianie nagrań

Spotkania online synchroniczne

- Nagrywanie spotkania i udostepnianie nagrań

Spotkania stacjonarne, Praktyki

- Rejestracja obecności uczestników

Administrator

- Wyświetlanie listy zgłoszonych problemów i możliwość odpowiadania klientom

- Dostęp do szczegółowych informacji o wszystkich użytkownikach systemu
- Dodawanie, usuwanie, modyfikowanie jednostek szkoleniowych i form kształcenia
- Dodawanie i usuwanie użytkowników (Nauczyciel, Dyrektor, Księgowy)
- Zarządzanie rolami
- "awaryjne" zarzadzanie uprawnieniami i kontami Klientów

Księgowy

- Raporty finansowe – zestawienie przychodów dla każdego webinaru/kursu/studium.
- Raport Lista „dłużników" – osoby, które skorzystały z usług, ale nie uiściły opłat.

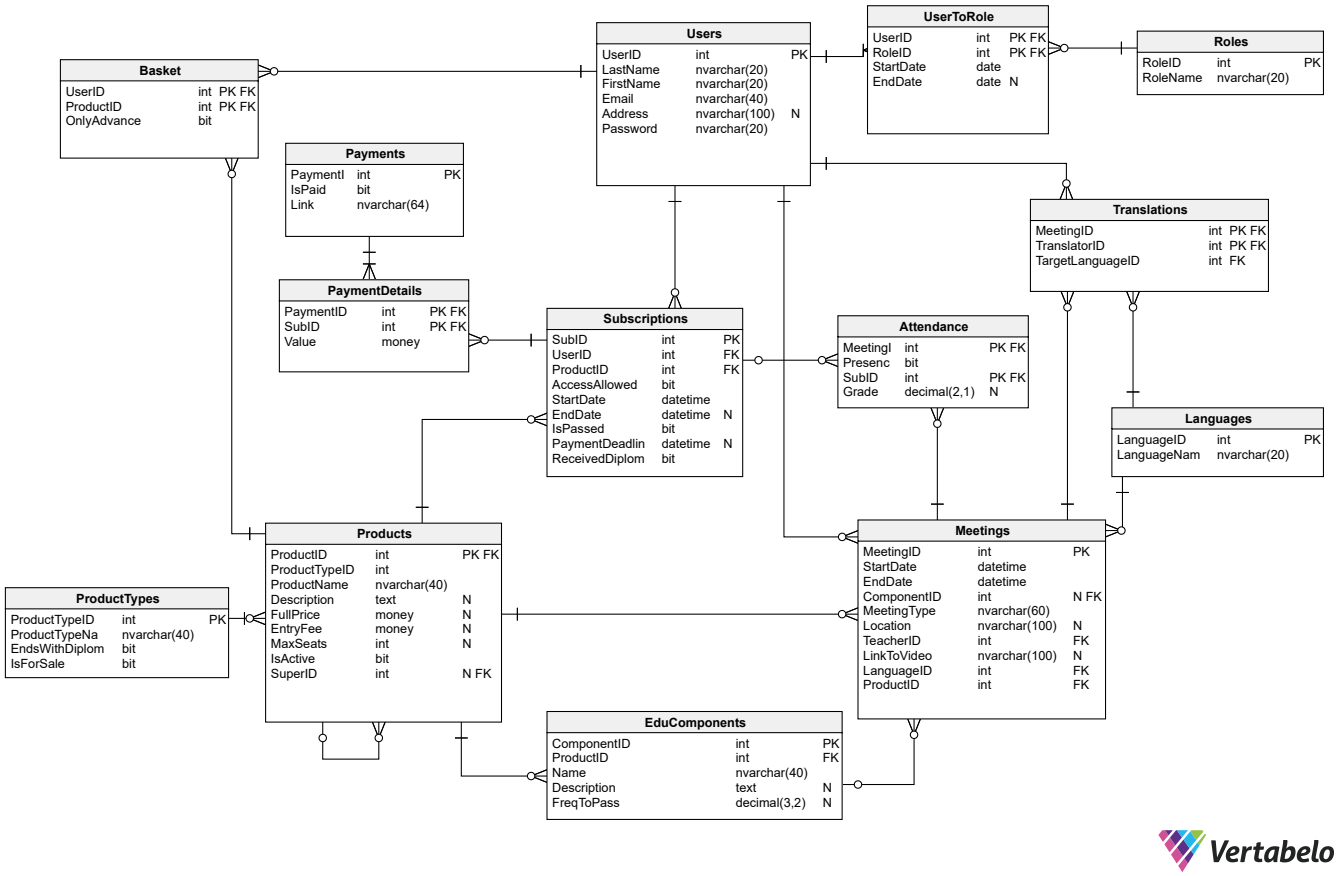
Dyrektor

Dziedziczy funkcje księgowego. Ponadto posiada dostęp do następujących funkcji:

- zarządzanie zgodami na płatność odroczoną
- zarządznie dyplomami - wydruki oraz rejestracja wydania dyplomu
- Ogólny raport dotyczący liczby zapisanych osób na przyszłe wydarzenia (z informacją, czy wydarzenie jest stacjonarnie, czy zdalnie).
- Ogólny raport dotyczący frekwencji na zakończonych już wydarzeniach.
- Lista obecności dla każdego szkolenia z datą, imieniem, nazwiskiem i informacją czy uczestnik był obecny, czy nie.
- Raport bilokacji: lista osób, które są zapisane na co najmniej dwa przyszłe szkolenia, które ze sobą kolidują czasowo.
- Dostęp do informacji o świadczonych usługach
- Dostęp do szczegółowych informacji o wszystkich użytkownikach systemu

2. Baza danych

Schemat bazy danych



Opis poszczególnych tabel

1. Users W tabeli users znajują się informacje o wszystkich użytkownikach systemu. Pola:

- UserID (PK) - unikalne ID użytkownika systemu
- LastName - Nazwisko użytkownika
- FirstName - Imię użytkownika
- Email - email, używany do logowania do systemu
- Address - adres użytkownika
- Password - hasło, używane do logowania do systemu

```
CREATE TABLE Users (
    UserID int IDENTITY(1,1)  NOT NULL,
    LastName nvarchar(20)  NOT NULL,
    FirstName nvarchar(20)  NOT NULL,
    Email nvarchar(40)  NOT NULL UNIQUE,
    Address nvarchar(100)  NULL,
    Password nvarchar(20) CHECK (LEN(Password) BETWEEN 8 AND 20)  NOT NULL,
    CONSTRAINT Users_pk PRIMARY KEY  (UserID)
);
```

2. Roles Tablica Roles zawiera informacje o rolach w systemie Pola:

- RoleID (PK) - ID unikalne dla każdej roli
- RoleName - nazwa roli w postaci napisu

```sql
CREATE TABLE Roles (
    RoleID int IDENTITY(1,1)  NOT NULL,
    RoleName nvarchar(20)  NOT NULL,
    CONSTRAINT Roles_pk PRIMARY KEY  (RoleID)
);
```

3. UserToRole Tabela UserToRole pełnii funkcje tabeli pośredniej, łączącej tabele Users i Roles. Dzięki tej tabeli realizujemy relację wiele do wiele tzn. jeden użytkownik może mieć wiele ról i wielu użytkowników może mieć tą samą rolę. Pola:

- UserID (FK) - ID użytkownika
- RoleID (FK) - ID roli
- StartDate - data rozpoczęcia pełnienia danej roli
- EndDate - data zakończenia pełnienia danej roli

```sql
CREATE TABLE UserToRole (
    UserID int  NOT NULL,
    RoleID int  NOT NULL,
    StartDate date  NOT NULL DEFAULT cast(getdate() as date),
    EndDate date  NULL,
    CONSTRAINT UserToRole_pk PRIMARY KEY  (UserID,RoleID)
);

ALTER TABLE UserToRole ADD CONSTRAINT User_To_Role_Role
    FOREIGN KEY (RoleID)
    REFERENCES Roles (RoleID);

ALTER TABLE UserToRole ADD CONSTRAINT User_To_Role_User
    FOREIGN KEY (UserID)
    REFERENCES Users (UserID);
```

4. Subscriptions Zawiera informacje o wykupionych dostępach do produktów. Pola:

- SubID (PK) - unikalne ID danej subskrypcji
- UserID (FK) - ID użytkownika
- ProductID (FK) - ID produktu
- AccessAllowed - czy użytkownik posiada dostęp do spotkań w ramach subskrypcji (przykładowo, jeżeli użytkownik nie zapłacił za zjazd i nie ma zgody na płatność odroczoną, to nie ma dostępu do spotkań w ramach tego zjazdu, pomimo że subskrypcja istnieje od czasu wpłaty zaliczki za studia)
- StartDate - data przyznania dostępu do produktu
- EndDate - data zabrania dostępu do produktu
- IsPassed - stan zaliczenia danego produktu (jeżeli nie dotyczny null)
- PaymentDate - Informacja do kiedy użytkownik musi zapłacić. Dyrektor może przedłużyć datę w ramach zgody na płatność odroczoną.
- RecivedDiploma - stan odebrania dyplomu

```sql
CREATE TABLE Subscriptions (
    SubID int IDENTITY(1,1)  NOT NULL,
    UserID int  NOT NULL,
    ProductID int  NOT NULL,
    AccessAllowed bit  NOT NULL,
    StartDate datetime  NOT NULL DEFAULT getdate(),
    EndDate datetime  NULL,
    IsPassed bit  NOT NULL DEFAULT 0,
    PaymentDeadline datetime  NULL,
    ReceivedDiploma bit  NOT NULL DEFAULT 0,
    CONSTRAINT Subscriptions_pk PRIMARY KEY  (SubID)
);

ALTER TABLE Subscriptions ADD CONSTRAINT Participants_Products
    FOREIGN KEY (ProductID)
    REFERENCES Products (ProductID);

ALTER TABLE Subscriptions ADD CONSTRAINT Participants_Users
    FOREIGN KEY (UserID)
    REFERENCES Users (UserID);
```

5. Products

Zawiera informacje o produktach oferowanych przez firmę. Produkty mogą obejmować webinary, studia, kursy, semestry, zjazdy i spotkania studyjne. Struktura jest hierarchiczna - np. semestry są podproduktami studiów, a zjazdy podproduktami semestrów. Spotkania w tabelach Meetings i MeetingsAssingments należą do produktów znajdujących się najniżej w hierarchii. Pola:

- ProductID (PK) - unikalne ID produktu.
- ProductTypeID - ID typu produktu. Szczegółowe informacje w tabeli ProductTypes.
- ProductName - nazwa produktu.
- Description - szczegółowy opis produktu.
- FullPrice - pełna cena produktu.
- EntryFee - opłata wstępna (jeśli obowiązuje).
- MaxSeats - maksymalna liczba miejsc dostępnych dla danego produktu.
- IsActive - informacja, czy produkt jest aktywny (bit).
- SuperID (FK) - ID nadrzędnego produktu (np. zjazd jest częścią studiów).

```sql
CREATE TABLE Products (
    ProductID int IDENTITY(1,1)  NOT NULL,
    ProductTypeID int  NOT NULL,
    ProductName nvarchar(40)  NOT NULL,
    Description text  NULL,
    FullPrice money  NULL,
    EntryFee money  NULL,
    MaxSeats int  NULL,
    IsActive bit  NOT NULL,
    SuperID int  NULL,
```

```
    Alias nvarchar(100) NOT NULL UNIQUE,
    CONSTRAINT Products_pk PRIMARY KEY  (ProductID)
);

ALTER TABLE Products ADD CONSTRAINT Products_ProductTypes
    FOREIGN KEY (ProductTypeID)
    REFERENCES ProductTypes (ProductTypeID);

ALTER TABLE Products ADD CONSTRAINT Products_Products
    FOREIGN KEY (SuperID)
    REFERENCES Products (ProductID);
```

6. Product Types Zawiera szczegółowe informacje o poszczególnych typach produktów (webinary, studia, kursy, zjazdy, spotkania studyjne, egzaminy i praktyki). Pola:

- ProductTypeID (PK) - unikalne ID typu
- ProductTypeName - nazwa typu produktu (np. webinar, studia, itd.)
- EndsWithDiploma - informacja czy dany typ produktu kończy się uzyskaniem dyplomu
- IsForSale - informacja czy dany typ można kupić/zasubskrybować (np. dla webinarów płatnych i darmowych IsForSale będzie oznaczone jako True, ale nie można zapisać się na pojedynczy semestr na studia - wtedy IsForSale jest False)

```
CREATE TABLE ProductTypes (
    ProductTypeID int  NOT NULL,
    ProductTypeName nvarchar(40)  NOT NULL,
    EndsWithDiploma bit  NOT NULL,
    IsForSale bit  NOT NULL,
    CONSTRAINT ProductTypes_pk PRIMARY KEY  (ProductTypeID)
);
```

7. Basket Zawiera produkty znajdujące się w koszykach poszczególnych klientów. wraz z informacją czy klient chce zapłacić pełną cenę czy tylko zaliczkę (jeżeli dotyczy). Pola:

- UserID - ID użytkownika
- ProductID - ID Produktu
- OnlyAdvance - Informacja czy klient chce zapłacić pełną kwotę za produkt czy tylko zaliczkę (jeżeli dotyczy). Wartość domyślna wynosi 0. Klient może zmienić na 1 tylko dla produktów, które mają możliwość wpłaty zaliczki.

```
CREATE TABLE Basket (
    UserID int  NOT NULL,
    ProductID int  NOT NULL,
    OnlyAdvance bit  NOT NULL DEFAULT 0,
    CONSTRAINT Basket_pk PRIMARY KEY  (UserID,ProductID)
);

ALTER TABLE Basket ADD CONSTRAINT Basket_Products
    FOREIGN KEY (ProductID)
```

```
    REFERENCES Products (ProductID);

ALTER TABLE Basket ADD CONSTRAINT Basket_Users
    FOREIGN KEY (UserID)
    REFERENCES Users (UserID);
```

8. Payments

Zawiera dane dotyczące płatności realizowanych w systemie. Pola:

- PaymentID (PK) - unikalne ID płatności.
- IsPaid - informacja o statusie płatności - czy zapłacono
- Link - odnośnik do potwierdzenia płatności (np. URL faktury).

```
CREATE TABLE Payments (
    PaymentID int IDENTITY(1,1)  NOT NULL,
    IsPaid bit  NOT NULL,
    Link nvarchar(64)  NOT NULL,
    CONSTRAINT Payments_pk PRIMARY KEY  (PaymentID)
);
```

9. PaymentDetails

Łączy płatności z subskrypcjami, przechowując szczegóły dotyczące wartości płatności. Pola:

- PaymentID (FK) - ID płatności.
- SubID (FK) - ID subskrypcji, której dotyczy płatność.
- Value - kwota płatności.

```
CREATE TABLE PaymentDetails (
    PaymentID int  NOT NULL,
    SubID int  NOT NULL,
    Value money  NOT NULL,
    CONSTRAINT PaymentDetails_pk PRIMARY KEY  (PaymentID,SubID)
);

ALTER TABLE PaymentDetails ADD CONSTRAINT Payment_Recon_Payments
    FOREIGN KEY (PaymentID)
    REFERENCES Payments (PaymentID);

ALTER TABLE PaymentDetails ADD CONSTRAINT Payment_Recon_Subscriptions
    FOREIGN KEY (SubID)
    REFERENCES Subscriptions (SubID);
```

10. Meetings

Zawiera informacje o spotkaniach (np. wykładach, zajęciach, egzaminach). Pola:

- MeetingID (PK) - unikalne ID spotkania.
- StartDate - data rozpoczęcia spotkania.
- EndDate - data zakończenia spotkania.
- ComponentID (FK) - ID komponentu edukacyjnego, do którego należy spotkanie.
- Location - miejsce, w którym odbywa się spotkanie. Może to być adres lub link do platformy chmurowej w przypadku spotkań online.
- MeetingType - typ spotkania (np. online, stacjonarne).
- LinkToVideo - odnośnik do nagrania spotkania (jeśli istnieje).
- TeacherID (FK) - ID nauczyciela prowadzącego spotkanie.
- LanguageID (FK) - język, w jakim odbywa się spotkanie.
- MaxSeats - maksymalna liczba uczestników.

```
CREATE TABLE Meetings (
    MeetingID int  NOT NULL,
    StartDate datetime  NOT NULL,
    EndDate datetime  NOT NULL,
    ComponentID int NULL,
    MeetingType nvarchar(60)  NOT NULL
        CHECK (MeetingType in ('Egzamin', 'Spotkanie online asynchroniczne',
'Spotkanie online synchroniczne','Praktyka','Spotkanie stacjonarne')),
    Location nvarchar(100)  NULL,
    TeacherID int  NOT NULL,
    LinkToVideo nvarchar(100)  NULL,
    LanguageID int  NOT NULL,
    CONSTRAINT Meetings_pk PRIMARY KEY  (MeetingID)
);

ALTER TABLE Meetings ADD CONSTRAINT Meetings_Languages
    FOREIGN KEY (LanguageID)
    REFERENCES Languages (LanguageID);

ALTER TABLE Meetings ADD CONSTRAINT FK_Meetings_Products
    FOREIGN KEY (ProductID)
    REFERENCES Products (ProductID);

ALTER TABLE Meetings ADD CONSTRAINT Meetings_Teachers
    FOREIGN KEY (TeacherID)
    REFERENCES Users (UserID);

ALTER TABLE Meetings ADD CONSTRAINT Meetings_edu_units
    FOREIGN KEY (ComponentID)
    REFERENCES EduComponents (ComponentID);
```

11. Attendance

Zawiera informacje o obecności użytkowników na spotkaniach. Pola:

- MeetingID (FK) - ID spotkania.
- SubID (FK) - ID subskrypcji użytkownika.
- Presence - informacja o obecności użytkownika na spotkaniu (bit).

- Grade - ocena uzyskana na spotkaniu. Dotyczy egzaminów - oprócz 100% frekwencji trzeba uzyskać ocenę pozytywną.

```
CREATE TABLE Attendance (
    MeetingID int  NOT NULL,
    Presence bit  NOT NULL,
    SubID int  NOT NULL,
    Grade decimal(2,1)  NULL CHECK (Grade in (2.0,3.0,3.5,4.0,4.5,5.0)),
    AbsenceMakeupMeetingID int NULL,
    CONSTRAINT Attendance_pk PRIMARY KEY  (MeetingID,SubID)
);

ALTER TABLE Attendance ADD CONSTRAINT Presence_Meetings
    FOREIGN KEY (MeetingID)
    REFERENCES Meetings (MeetingID);

ALTER TABLE Attendance ADD CONSTRAINT Presence_Subscriptions
    FOREIGN KEY (SubID)
    REFERENCES Subscriptions (SubID);
```

12. EduComponents

Zawiera dane o komponentach edukacyjnych, które są częścią produktów. Przykładowo komponentem jest przedmiot na studiach, który należy do semestru i może pojawiać się na wielu zjazdach, lub moduł, który należy do kursu. Komponentami mogą być również egzaminy i praktyki. Komponenty grupują spotkania tematycznie i dodatkowo mają osobne zasady zaliczenia na podstawie obecności. Są potrzebne do wyświetlania sylabusa.

Pola:

- ComponentID (PK) - unikalne ID komponentu edukacyjnego.
- ProductID (FK) - ID produktu, do którego należy komponent
- Name - nazwa komponentu edukacyjnego.
- Description - szczegółowy opis komponentu.
- FreqToPass - wymagane minimum zaliczeń (np. liczba spotkań do zaliczenia).

```
CREATE TABLE EduComponents (
    ComponentID int IDENTITY(1,1)  NOT NULL,
    ProductID int  NOT NULL,
    Name nvarchar(40)  NOT NULL,
    Description text  NULL,
    FreqToPass decimal(3,2)  NULL,
    CONSTRAINT EduComponents_pk PRIMARY KEY  (ComponentID)
);

ALTER TABLE EduComponents ADD CONSTRAINT EduComponents_Products
    FOREIGN KEY (ProductID)
    REFERENCES Products (ProductID);
```

13. Languages

Przechowuje listę dostępnych języków w systemie. Pola:

- LanguageID (PK) - unikalne ID języka.
- LanguageName - nazwa języka (np. angielski, polski).

```
CREATE TABLE Languages (
    LanguageID int IDENTITY(1,1)  NOT NULL,
    LanguageName nvarchar(20)  NOT NULL,
    CONSTRAINT Languages_pk PRIMARY KEY  (LanguageID)
);
```

14. Translations

Przechowuje informacje o dostępnych tłumaczeniach dla spotkań. Pola:

- MeetingID (FK) - ID spotkania.
- TranslatorID (FK) - ID tłumacza.
- TargetLanguageID (FK) - ID języka docelowego tłumaczenia.

```
CREATE TABLE Translations (
    MeetingID int  NOT NULL,
    TranslatorID int  NOT NULL,
    TargetLanguageID int  NOT NULL,
    CONSTRAINT Translations_pk PRIMARY KEY  (MeetingID,TranslatorID)
);

ALTER TABLE Translations ADD CONSTRAINT Translations_Languages
    FOREIGN KEY (TargetLanguageID)
    REFERENCES Languages (LanguageID);

ALTER TABLE Translations ADD CONSTRAINT Translations_Meetings
    FOREIGN KEY (MeetingID)
    REFERENCES Meetings (MeetingID);

ALTER TABLE Translations ADD CONSTRAINT Translations_Users
    FOREIGN KEY (TranslatorID)
    REFERENCES Users (UserID);
```

Widoki

vProductFreeSeats - pokazuje wszystkie produkty wraz z aktualną liczbą wolnych miejsc (wszystkie role) [WN]

```
alter   VIEW [dbo].[vProductFreeSeats]
AS
SELECT
    p.Description,
```

```
    p.EntryFee,
    p.FullPrice,
    p.ProductName,
    pt.ProductTypeName,
    p.maxseats,
    dbo.freeseats(p.productid) AS FreeSeats
FROM
    Products p
    INNER JOIN ProductTypes pt ON p.ProductTypeID = pt.ProductTypeID
WHERE
    p.IsActive = 1
    and pt.IsForSale=1
```

debtors_list - lista dłużników, czyli osób, które skorzystały z usług, ale nie uiściły opłat (Dyrektor, Księgowy) [KN]

```
create view debtors_list
as
select u.UserID,u.FirstName,u.LastName,u.Email,u.Address, GETDATE()-
s.PaymentDeadline as PaymentDelay, pd.Value
from Users u
join Subscriptions s
on s.userid=u.UserID
join Attendance a
on a.SubID=s.SubID
join PaymentDetails pd
on pd.SubID=s.SubID
join Payments p
on p.PaymentID = pd.PaymentID
where p.IsPaid=0 and a.Presence=1
and GETDATE()>s.PaymentDeadline
```

vTeachers - wypisuje wszystkich nauczycieli (Dyrektor, Administrator) [WN]

```
CREATE VIEW [dbo].[vTeachers]
AS
SELECT     u.UserID, u.LastName, u.FirstName
FROM        dbo.Users AS u INNER JOIN
                  dbo.UserToRole AS ur ON u.UserID = ur.UserID INNER JOIN
                  dbo.Roles AS r ON ur.RoleID = r.RoleID
WHERE     (r.RoleName = 'Nauczyciel')
```

vTranslators - wypisuje wszystkich tłumaczy (Dyrektor, Administrator) [KN]

```
ALTER   VIEW [dbo].[vTranslators]
AS
```

```
SELECT     u.UserID, u.LastName, u.FirstName
FROM        dbo.Users AS u INNER JOIN
                  dbo.UserToRole AS ur ON u.UserID = ur.UserID INNER JOIN
                  dbo.Roles AS r ON ur.RoleID = r.RoleID
WHERE     (r.RoleName='Translator')
```

vClients - wypisuje wszystkich klientów (Dyrektor, Administrator, Księgowy) [KN]

```
ALTER   VIEW [dbo].[vClients]
AS
SELECT     u.UserID, u.LastName, u.FirstName
FROM        dbo.Users AS u INNER JOIN
                  dbo.UserToRole AS ur ON u.UserID = ur.UserID INNER JOIN
                  dbo.Roles AS r ON ur.RoleID = r.RoleID
WHERE     (r.RoleName = 'Klient')
```

v_users_roles - wypisuje użytkowników i ich role w systemie (id użytkownika, imię, nazwisko, rola) (Dyrektor, Administrator) [KN]

```
create view v_users_roles
as
select u.userid, FirstName, LastName, RoleName
from Users u join UserToRole utr
on utr.UserID=u.UserID
join Roles r
on r.RoleID=utr.RoleID
```

BilocationReport - lista osób, które są zapisane na co najmniej dwa przyszłe szkolenia, które ze sobą kolidują czasowo.4 (Dyrektor) [KN]

```
CREATE VIEW BilocationReport
AS
SELECT userid, FirstName, LastName, Address, Email
FROM Users u
WHERE EXISTS (
    SELECT 1
    FROM dbo.StudentsBilocation(u.UserID)
);
```

financialReport - zestawienie przychodów dla każdego webinaru/kursu/studium (Dyrektor, Księgowy) [KN]

```
alter view [dbo].[financialReport]
as
select p.ProductID,p.ProductName, dbo.GetProductTypeName(p.ProductID) as
ProductType ,sum(pd.value) as TotalRevenues, dbo.productstartdate(p.productID) as
```

```
StartDate
from products p
join Subscriptions s
on s.ProductID=p.ProductID
join PaymentDetails pd
on pd.SubID=s.SubID
join ProductTypes pt
on pt.ProductTypeID=p.ProductTypeID
where ProductTypeName in ('kurs','webinar','studia','spotkanie studyjne')
group by p.ProductID,p.ProductName
```

ClientsWaitingForDiploma - lista klientów i ich adresów korespondencyjnych, którzy powinni dostać dyplom, a nie dostali jeszcze. (Dyrektor) [KN]

```
CREATE view [dbo].[ClientsWaitingForDiploma]
as
select u.UserID, FirstName, LastName, Address, s.SubID, p.ProductName
from Users u
join Subscriptions s
on s.UserID=u.UserID
join Products p
on p.ProductID=s.ProductID
where dbo.GetProductTypeName(s.ProductID) in ('kurs','studia')
and IsPassed=1
and ReceivedDiploma=0
```

Funkcje

freeseats - pokazuje liczbę wolnych miejsc dla produktu o podanym ID (wszystkie role) [WN]

```
ALTER   FUNCTION [dbo].[FreeSeats]
(
    @p_productid INT
)
RETURNS INT
AS
BEGIN
    RETURN (
        SELECT
            MAX(p.MaxSeats) - COUNT(s.ProductID)
        FROM
            Products p
        LEFT JOIN
            Subscriptions s
            ON (dbo.GetFinalProductID(s.ProductID) = p.ProductID
            or dbo.GetFinalProductID(p.ProductID)=s.ProductID
            or p.ProductID=s.ProductID)
        WHERE
            p.ProductID = @p_productid
```

```
    );
END;
```

getProfits - pokazuje zyski ze sprzedaży produktów dla podanego okresu (Dyrektor, Księgowy) [WN]

```
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

ALTER   FUNCTION [dbo].[getProfits]
(
    @entryDate as date,
    @closeDate as date
)
RETURNS money
AS
BEGIN
    RETURN(select sum(p.value)
    from PaymentDetails p join Subscriptions s on p.SubID = s.SubID
    where s.StartDate between @entryDate and @closeDate)

END
```

GetUserBasket - wyświetla koszyk danego użytkownika (Klient) [KN]

```
CREATE FUNCTION GetUserBasket (@UserID INT)
RETURNS TABLE
AS
RETURN
(
    SELECT
        b.ProductID,
        p.ProductName,
        pt.ProductTypeName,
        case
            when b.OnlyAdvance = 1 THEN p.EntryFee
            ELSE p.FullPrice
        END AS Price
        ,
        b.OnlyAdvance
    FROM
        Basket b
    INNER JOIN Products p ON b.ProductID = p.ProductID
    INNER JOIN ProductTypes pt ON p.ProductTypeID = pt.ProductTypeID
    WHERE
        b.UserID = @UserID
);
```

GetProductsByID - wypisuje ID produktu i wszystkich podproduktów (również podpodproduktów). Przydatne do tworzenia subskrypcji dla zakupionego produktu. [KN]

```
CREATE FUNCTION GetProductsByID (@ProductID INT)
RETURNS TABLE
AS
RETURN
(
    WITH ProductsLevel1 AS
    (
        SELECT
            p.ProductID
        FROM
            Products p
        WHERE
            p.ProductID = @ProductID
    ),
    ProductsLevel2 AS
    (
        SELECT
            p.ProductID
        FROM
            Products p
        INNER JOIN ProductsLevel1 l1 ON p.SuperID = l1.ProductID
    ),
    ProductsLevel3 AS
    (
        SELECT
            p.ProductID
        FROM
            Products p
        INNER JOIN ProductsLevel2 l2 ON p.SuperID = l2.ProductID
    )
    SELECT *
    FROM ProductsLevel1
    UNION
    SELECT *
    FROM ProductsLevel2
    UNION
    SELECT *
    FROM ProductsLevel3
);
```

GetProductTypeName - zwraca typ produktu dla produktu o podanym ID (funkcja pomocnicza) [KN]

```
ALTER FUNCTION [dbo].[GetProductTypeName] (@ProductID INT)
RETURNS NVARCHAR(40)
AS
BEGIN
```

```
    DECLARE @ProductTypeName NVARCHAR(40);

    SELECT
        @ProductTypeName = pt.ProductTypeName
    FROM
        Products p
    INNER JOIN
        ProductTypes pt ON p.ProductTypeID = pt.ProductTypeID
    WHERE
        p.ProductID = @ProductID;

    RETURN @ProductTypeName;
END;
```

GetMeetingsSchedule - zwraca tabelę z przyszłymi spotkaniami na które użytkownik jest zapisany. (Klient, Dyrektor) [KN]

```
CREATE FUNCTION GetMeetingsSchedule (@UserID INT)
RETURNS TABLE
AS
RETURN
(
    SELECT MeetingID, m.StartDate, m.EndDate, ProductName
    FROM Meetings m
    JOIN Products p ON p.ProductID = dbo.GetFinalProductID(m.ProductID)
    JOIN Subscriptions s ON s.ProductID = p.ProductID
    WHERE s.UserID = @UserID
);
```

StudentsBilocation - raport bilokacji konkretnego użytkownika (tabela przyszłych spotkań na które użytkownik jest zapisany i kolidują ze sobą czasowo) (Klient, Dyrektor) [KN]

```
create function StudentsBilocation(@UserID int)
returns table
as
return(
select distinct s1.* from dbo.getMeetingsSchedule(@UserID) s1
join dbo.getMeetingsSchedule(@UserID) s2
on s1.startdate between s2.startdate and s2.enddate
and s1.meetingId!=s2.meetingId)
```

GetFinalProductID - po podaniu id produktu rekurencyjnie szuka najstarszego przodka (produkt którego supreID jest NULL) [WN]

```
ALTER    FUNCTION [dbo].[GetFinalProductID]
(
    @prodID int
```

```
)
RETURNS int
AS
BEGIN

    DECLARE @result int, @temp int

    SELECT @result = @prodID

    SELECT @temp = p.SuperID
    FROM Products p
    WHERE p.ProductID = @result

    WHILE @temp is not NULL
    BEGIN
        set @result = @temp

        SELECT @temp = p.SuperID
        FROM Products p
        WHERE p.ProductID = @result
    END

    RETURN @result
END
```

productStartDate - zwraca datę pierwszego spotkania w ramach danego produktu. [KN]

```
create function productStartDate(@ProductID int)
returns datetime
as
begin
return(
select min(startdate) from Meetings m
where dbo.GetFinalProductID(m.ProductID)=@ProductID)
end
```

getUsersSubscriptions - zwraca tabelę z subskrypcjami danego użutkownika i informacją o stanie zaliczenia/otrzymania dyplomu (Klient) [KN]

```
create function getUsersSubscriptions(@UserId int)
returns table
as
return
(select s.SubID, s.ProductID, ProductName, AccessAllowed, PaymentDeadline,
IsPassed, ReceivedDiploma
from Subscriptions s
join Products p
on p.ProductID=s.ProductID
where userid=@UserId)
```

availableTranslations - zwraca tabelę z dostępnymi tłumaczeniami na danym spotkaniu. (wszyscy użytkownicy) [KN]

```
create function availableTranslations(@MeetingID int)
returns table
as
return(
select targetLanguageID, LanguageName, TranslatorID, FirstName, LastName
from Translations t
join Users u
on u.UserID=t.TranslatorID
join Languages l
on l.LanguageID=t.TargetLanguageID
where MeetingID=@MeetingID
)
```

areExamsPassed - sprawdza czy wszystkie egzaminy w ramach danego przedmiotu są zaliczone [WN]

```
ALTER FUNCTION [dbo].[areExamsPassed]
(
    -- Add the parameters for the function here
    @userID int, @compID int

)
RETURNS bit
AS
BEGIN
    -- Declare the return variable here
    DECLARE @result bit

    -- Add the T-SQL statements to compute the return value here
    SET @result = CASE
                  WHEN NOT EXISTS (
                      SELECT 1
                      FROM Attendance a
                      JOIN Subscriptions s ON s.SubID = a.SubID
                      JOIN Meetings m ON m.MeetingID = a.MeetingID
                      WHERE s.UserID = @userID
                      AND m.ComponentID = @compID
                      AND a.grade NOT IN (2, 0)
                  ) THEN 1
                  ELSE 0
                END;

    -- Return the result of the function
    RETURN @result

END
```

calculateFrequency - sprawdza czy klient spełnił wymagania zaliczenia dotyczące obecności [WN]

```
ALTER   FUNCTION [dbo].[calculateFrequency]
(
    -- Add the parameters for the function here
    @compID int, @userID int
)
RETURNS bit
AS
BEGIN
    -- Declare the return variable here
    DECLARE @result bit

    -- Add the T-SQL statements to compute the return value here
    DECLARE @allCnt int
    if (select count(*) from Meetings m
    join Attendance a on a.MeetingID = m.MeetingID
    join Subscriptions s on s.SubID = a.SubID
    where m.ComponentID = @compID and s.UserID = @userID) > 0
    begin


        SET @allCnt = (select count(*) from Meetings m
        join Attendance a on a.MeetingID = m.MeetingID
        join Subscriptions s on s.SubID = a.SubID
        where m.ComponentID = @compID and s.UserID = @userID and a.Presence = 1) /
        (select count(*) from Meetings m
        join Attendance a on a.MeetingID = m.MeetingID
        join Subscriptions s on s.SubID = a.SubID
        where m.ComponentID = @compID and s.UserID = @userID)

        SET @result = CASE
                      WHEN @allCnt >= ISNULL((SELECT FreqToPass FROM EduComponents
WHERE ComponentID = @compID), 0)
                      THEN 1
                      ELSE 0
                  END;
    end
    else
    begin
        set @result = 1
    end
    -- Return the result of the function
    RETURN @result

END
```

Triggery

validateMeetingProduct - uniemożliwia wpisanie spotkania przypisanego do produktu, który nie należy do produktu z tabeli EduComponents dla danego ComponentID. Przykładowo, można wpisać spotkanie w ramach kursu, tylko jeżeli komponent również należy do tego kursu. W przypadku studiów produkt może być zjazdem, a komponent może należeć do semestru, w ramach którego odbywa się ten zjazd. [KN]

```
ALTER TRIGGER [dbo].[validateMeetingProduct]
ON [dbo].[Meetings]
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM inserted i
        join Products p on p.ProductID=i.ProductID
        join EduComponents ec on ec.ComponentID=i.ComponentID
        where i.ComponentID is not null
        and (p.ProductID!=ec.ProductID and p.SuperID is not null and
p.SuperID!=ec.ProductID)
        or (p.SuperID is null and p.ProductID!=ec.ProductID)
    )
    BEGIN
        RAISERROR ('ProductID w tabeli Meetings nie należy do ProductID z
EduComponents dla danego ComponentID', 16, 1);
        ROLLBACK TRANSACTION;
    END
END;
```

only_translators - uniemożliwia wpisania jako tłumacza użytkownika, który nie pełni tej roli w systemie [KN]

```
create trigger only_translators
on dbo.Translations
after insert, update
as
begin
    if not exists
    (select 1 from vTranslators t
    join inserted i
    on i.TranslatorID=t.UserID)
    BEGIN
        RAISERROR ('Podany użytkownik nie jest translatorem', 16, 2);
        ROLLBACK TRANSACTION;
    END
end
```

check_teacher - uniemożliwia wpisania jako nauczyciela użytkownika, który nie pełni tej roli w systemie. Sprawdza czy podany nauczyciel nie prowadzi w tym czasie innego spotkania. (wyjątek - studia i spotkanie studyjne) [KN]

```
ALTER trigger [dbo].[check_teacher]
on [dbo].[Meetings]
after insert, update
as
begin
    if not exists
    (select 1 from vTeachers t
    join inserted i
    on i.TeacherID=t.UserID)
    BEGIN
        RAISERROR ('Podany użytkownik nie jest nauczycielem', 16, 3);
        ROLLBACK TRANSACTION;
    END
    if exists
    (select 1 from Meetings m
    join inserted i
    on i.TeacherID=m.TeacherID
    where ((i.StartDate<m.EndDate and i.StartDate>m.StartDate) or
(m.StartDate<i.EndDate and m.StartDate>i.StartDate))
    and ((select dbo.GetProductTypeName(i.productid))!='spotkanie studyjne' or
(select dbo.GetProductTypeName(m.productid))!='zjazd'))
        BEGIN
        RAISERROR ('Podany nauczyciel prowadzi w tym czasie inne spotkanie', 16,
4);
        ROLLBACK TRANSACTION;
    END

end
```

accesAllowed - Po ustawieniu pola AccessAllowed na 1, tworzy rekordy w tabeli Attendance, aby umożliwić uczestnictwo i rejestrowanie obecności na spotkaniach w ramach danej subskrypcji. [KN]

```
ALTER TRIGGER [dbo].[access_allowed]
   ON  [dbo].[Subscriptions]
   AFTER  UPDATE, insert
AS
if exists (
    select 1
    from inserted i
    left join deleted d on i.subid = d.subid
    where i.accessallowed = 1 and (d.accessallowed is null or d.accessallowed = 0)
)
BEGIN
    SET NOCOUNT ON
    insert into Attendance (MeetingId, SubID)
    (select m.MeetingID, SubID
    from Products p
    join Meetings m
    on p.ProductID=m.ProductID
    join inserted i
```

```
    on (i.ProductID=dbo.GetFinalProductID(p.ProductID) or i.ProductID=p.ProductID)
    and not exists(select 1 from Attendance a where (a.SubID=i.SubID and
a.MeetingID=m.MeetingID))
    )

END
```

paymentcheck - po opłaceniu pełnej kwoty zamówienia przyznawany jest dostęp do danej subskrypcji. Jeżeli zapłacono zaliczkę za studia, tworzone są osobne subskrypcje i płatności do poszczególnych zjazdów. Jeżeli zapłacono zaliczkę za kurs, tworzona jest nowa płatność do tej samej subskrypcji na pozostałą kwotę. Terminy wymaganej płatności są odpowiednio aktualizowane. [KN]

```
ALTER   TRIGGER [dbo].[paymentcheck]
ON [dbo].[Payments]
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1
        FROM Inserted i
        JOIN Deleted d ON i.PaymentID = d.PaymentID
        WHERE i.IsPaid = 1 AND d.IsPaid = 0
    )
    BEGIN
        DECLARE @SubID INT, @UserID INT, @ProductID INT, @ProductTypeName
NVARCHAR(40), @EntryFee MONEY, @FullPrice MONEY, @Value Money;

        DECLARE cur CURSOR FOR
        SELECT s.SubID, s.UserID, s.ProductID, (select
dbo.GetProductTypeName(s.ProductID)), p.EntryFee, p.FullPrice,pd.Value
        FROM Inserted i
        join PaymentDetails pd
        on pd.PaymentID=i.PaymentID
        JOIN Subscriptions s ON pd.SubID = s.SubID
        JOIN Products p ON s.ProductID = p.ProductID
        WHERE i.IsPaid = 1;

        OPEN cur;
        FETCH NEXT FROM cur INTO @SubID, @UserID, @ProductID, @ProductTypeName,
@EntryFee, @FullPrice,@Value;

        WHILE @@FETCH_STATUS = 0
        BEGIN
            IF @Value=@FullPrice or @Value=@FullPrice-@EntryFee
            BEGIN
                UPDATE Subscriptions
                SET AccessAllowed = 1
                WHERE SubID = @SubID;
```

```
            END
            ELSE
            BEGIN
                IF @ProductTypeName = 'studia'
                BEGIN
                    INSERT INTO Subscriptions (UserID,
ProductID,AccessAllowed,PaymentDeadline)
                    SELECT distinct @UserID, p.ProductID,0,
DATEADD(dd,-3,dbo.FindStartDate(p.ProductID))
                    FROM Products p
                    where dbo.GetFinalProductID(p.ProductID)=@ProductID and
p.ProductID!=@ProductID and p.FullPrice is not null

                    DECLARE @Subscriptions TABLE (SubID INT, ProductID INT);
                    INSERT INTO @Subscriptions (SubID, ProductID)
                    SELECT distinct s.SubID, s.ProductID
                    FROM Subscriptions s
                    where dbo.GetFinalProductID(s.ProductID)=@ProductID and
s.ProductID!=@ProductID and UserID=@UserID

                    DECLARE @NewSubID INT, @NewProductID INT;
                    DECLARE sub_cursor CURSOR FOR
                    SELECT SubID, ProductID FROM @Subscriptions;

                    OPEN sub_cursor;
                    FETCH NEXT FROM sub_cursor INTO @NewSubID, @NewProductID;

                    WHILE @@FETCH_STATUS = 0
                    BEGIN
                        DECLARE @PaymentLink NVARCHAR(64);
                        SET @PaymentLink = CONCAT('https://payment.', NEWID(),
'.pl');

                        INSERT INTO Payments (IsPaid, Link)
                        VALUES (0, @PaymentLink);
                        DECLARE @PaymentID INT = SCOPE_IDENTITY();

                        INSERT INTO PaymentDetails (PaymentID, SubID, Value)
                        VALUES (@PaymentID, @NewSubID,
                                (SELECT FullPrice FROM Products WHERE ProductID =
@NewProductID));

                        FETCH NEXT FROM sub_cursor INTO @NewSubID, @NewProductID;
                    END;

                    CLOSE sub_cursor;
                    DEALLOCATE sub_cursor;

        UPDATE Subscriptions
                SET PaymentDeadline=Null
                WHERE SubID = @SubID;
        END

            ELSE IF @ProductTypeName = 'kurs'
```

```
            BEGIN
            DECLARE @PaymentLink2 NVARCHAR(64);
        SET @PaymentLink2 = CONCAT('https://payment.', NEWID(), '.pl');
        INSERT INTO Payments (IsPaid, Link)
            VALUES (0, @PaymentLink2);
        DECLARE @PaymentID2 INT = SCOPE_IDENTITY();

        insert into PaymentDetails (PaymentID,SubID,Value)
            Values(@PaymentID2,@SubID,@FullPrice-@EntryFee);

            update Subscriptions
            set PaymentDeadline=DATEADD(dd,-3,dbo.FindStartDate(@ProductID))
            where SubID=@SubID
                END
            END

            FETCH NEXT FROM cur INTO @SubID, @UserID, @ProductID,
@ProductTypeName, @EntryFee, @FullPrice,@Value;
        END;

        CLOSE cur;
        DEALLOCATE cur;
    END
END;
```

CheckIfEnrolled - uniemożliwia wprowadzeniu obecności osobie nie zapisanej na produkt do którego należy dane spotkanie [WN]

```
ALTER TRIGGER [dbo].[CheckIfEnrolled]
ON [dbo].[Attendance]
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN Subscriptions s ON i.SubID = s.SubID
        JOIN Meetings m ON m.MeetingID = i.MeetingID
        JOIN Products p ON p.ProductID = m.ProductID
        WHERE dbo.GetFinalProductID(s.ProductID) !=
dbo.GetFinalProductID(p.ProductID)
    )
    BEGIN
        RAISERROR('User NOT enrolled for meeting', 16, 1);
        ROLLBACK;
    END
END;
GO
```

checkComponentsProduct - nie pozwala dodać do tabeli EduComponents komponentu należącego do innych produktów niż semstr lub kurs. [KN]

```
create trigger checkComponentsProduct
on EduComponents
after insert, update
as
begin
if exists (select 1 from inserted i where
dbo.GetProductTypeName(i.productID) in ('studia', 'zjazd', 'webinar','spotkanie
studyjne'))
begin
        THROW 50000, 'Komponent nie może należeć do tego produktu', 1;
        ROLLBACK TRANSACTION;
end
end
```

Procedury

getMeetingAttendance - zwraca tablicę obecności dla danego spotkania (Nauczyciel, Dyrektor) [WN]

```
ALTER   PROCEDURE [dbo].[getMeetingAttendance]
    @thisMeetingID INT
AS
BEGIN

    SET NOCOUNT ON;
    SELECT CONCAT(u.FirstName, ' ', u.LastName) as Name, a.Presence
    FROM Attendance a join Subscriptions s on s.SubID = a.SubID
    join Users u on u.UserID = s.UserID
    where a.MeetingID = @thisMeetingID

    SELECT avg(CAST(a.Presence as float)) as Attendance, a.MeetingID
    FROM Attendance a
    where a.MeetingID = @thisMeetingID
    group by a.MeetingID
END
```

getStudiesSyllabus - zwraca syllabus studiów (wszystkie role) [WN]

```
ALTER   PROCEDURE [dbo].[getStudiesSyllabus]
    @studyName NVARCHAR(MAX)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        ec.Name, ec.Description, p.ProductName
```

```
    FROM Products p
    JOIN Products sp ON p.SuperID = sp.ProductID
    JOIN EduComponents ec on p.ProductID = ec.ProductID
    JOIN ProductTypes pt on sp.ProductTypeID = pt.ProductTypeID
    WHERE sp.Alias = @studyName and pt.ProductTypeName = 'Studia'
    ORDER BY p.ProductName, ec.Name
END
```

getSyllabusSemester - zwraca syllabus dla danego semestru albo kursu (wszystkie role) [WN]

```
ALTER PROCEDURE [dbo].[getSyllabusSemester](
    @semesterName nvarchar(MAX) = ''
)
AS
BEGIN
    SET NOCOUNT ON;
SELECT
    ec.ComponentID,
    ec.Name AS Name,
    CAST(ec.Description as nvarchar(MAX)) AS Description,
    COUNT(m.MeetingID) AS MeetingCount
FROM
    EduComponents ec
JOIN Products p ON ec.ProductID = p.ProductID
JOIN ProductTypes pt ON p.ProductTypeID = pt.ProductTypeID
JOIN Meetings m ON m.ComponentID = ec.ComponentID
WHERE pt.ProductTypeName in ('Semestr','Kurs')
    AND p.Alias = @semesterName
GROUP BY ec.ComponentID, ec.Name, CAST(ec.Description as nvarchar(MAX))
ORDER BY CAST(ec.Name AS NVARCHAR(MAX));
END
```

getTotalUserAttendance - zwraca całą zapisaną obecność dla danego użytkownika (Dyrektor, Klient) [WN]

```
ALTER   PROCEDURE [dbo].[getTotalUserAttendance]
    -- Add the parameters for the stored procedure here
    @userName nvarchar(MAX) = '',
    @userID int = -1
AS
BEGIN
    -- SET NOCOUNT ON added to prevent extra result sets from
    -- interfering with SELECT statements.
    SET NOCOUNT ON;

    IF @userID = -1 AND dbo.studentNameToUserIdNum(@userName) > 1
    BEGIN
        throw 51000, 'Wielu uczniów o tym samym imieniu i nazwisku', 1;
    END
```

```
    IF @userID = -1
    begin
        SELECT ec.Name, m.MeetingID, a.Presence, CAST(a.Presence AS INT) as
IntCast
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on u.UserID = s.UserID
        join Meetings m on m.MeetingID = a.MeetingID
        join EduComponents ec on ec.ComponentID = m.ComponentID
        WHERE CONCAT(u.FirstName, ' ', u.LastName) = @userName
        order by ec.Name

        SELECT avg(CAST(a.Presence AS float)) AS Frequency
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on u.UserID = s.UserID
        WHERE CONCAT(u.FirstName, ' ', u.LastName) = @userName

    end
    else
    begin
        SELECT ec.Name, m.MeetingID, a.Presence, CAST(a.Presence AS INT) as
IntCast
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on u.UserID = s.UserID
        join Meetings m on m.MeetingID = a.MeetingID
        join EduComponents ec on ec.ComponentID = m.ComponentID
        WHERE u.UserID = @userID
        order by ec.Name

        SELECT avg(CAST(a.Presence AS float)) AS Frequency
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on u.UserID = s.UserID
        WHERE u.UserID = @userID
    end

END
```

getUserComponentAttendance - obecność danego użytkownika na danym przedmiocie (Nauczyciel, Dyrektor, Klient) [WN]

```
ALTER   PROCEDURE [dbo].[getUserComponentAttendance]
    -- Add the parameters for the stored procedure here
    @userName nvarchar(MAX) = '',
    @userID int = -1,
    @componentID int
AS
BEGIN
    -- SET NOCOUNT ON added to prevent extra result sets from
    -- interfering with SELECT statements.
    SET NOCOUNT ON;
```

```
    IF @userID = -1 AND dbo.studentNameToUserIdNum(@userName) > 1
    BEGIN
        throw 51000, 'Wielu uczniów o tym samym imieniu i nazwisku', 1;
    END

    -- Insert statements for procedure here
    IF @userID = -1
    begin
        SELECT m.MeetingID, m.StartDate, a.Presence
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on s.UserID = u.UserID
        join Meetings m on m.MeetingID = a.MeetingID
        join EduComponents ec on ec.ComponentID = m.ComponentID
        WHERE CONCAT(u.FirstName,' ',u.LastName) = @userName and ec.ComponentID =
@componentID
        ORDER BY m.StartDate

        SELECT avg(CAST(a.Presence as float)) as Frequency
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on s.UserID = u.UserID
        join Meetings m on m.MeetingID = a.MeetingID
        join EduComponents ec on ec.ComponentID = m.ComponentID
        WHERE CONCAT(u.FirstName,' ',u.LastName) = @userName and ec.ComponentID =
@componentID
    end
    else
    begin
        SELECT m.MeetingID, m.StartDate, a.Presence
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on s.UserID = u.UserID
        join Meetings m on m.MeetingID = a.MeetingID
        join EduComponents ec on ec.ComponentID = m.ComponentID
        WHERE u.UserID = @userID and ec.ComponentID = @componentID
        ORDER BY m.StartDate

        SELECT avg(CAST(a.Presence as float)) as Frequency
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on s.UserID = u.UserID
        join Meetings m on m.MeetingID = a.MeetingID
        join EduComponents ec on ec.ComponentID = m.ComponentID
        WHERE u.UserID = @userID and ec.ComponentID = @componentID
    end
END
```

getUserSubjectGrades - zwraca oceny danego użytkownika z danego przedmiotu wraz z detalami (Klient, Nauczyciel, Dyrektor) [WN]

```
ALTER   PROCEDURE [dbo].[getUserSubjectGrades]
    -- Add the parameters for the stored procedure here
    @userID int = -1,
    @userName nvarchar(MAX) = '',
```

```
    @subjectID nvarchar(MAX)

AS
BEGIN
    -- SET NOCOUNT ON added to prevent extra result sets from
    -- interfering with SELECT statements.
    SET NOCOUNT ON;

    IF @userID = -1 AND dbo.studentNameToUserIdNum(@userName) > 1
    BEGIN
        throw 51000, 'Wielu uczniów o tym samym imieniu i nazwisku', 1;
    END

    -- Insert statements for procedure here
    if @userID = -1
    begin
        SELECT CAST(m.StartDate as date) as 'Date', a.Grade as 'Grade',
m.MeetingType as 'Type'
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on u.UserID = s.UserID
        join Meetings m on m.MeetingID = a.MeetingID
        join EduComponents ec on ec.ComponentID = m.ComponentID
        WHERE CONCAT(u.FirstName, ' ', u.LastName) = @userName and a.Grade is not
NULL and
            ec.ComponentID = @subjectID
        ORDER BY m.StartDate
    end
    else
    begin
        SELECT CAST(m.StartDate as date) as 'Date', a.Grade as 'Grade',
m.MeetingType as 'Type'
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on u.UserID = s.UserID
        join Meetings m on m.MeetingID = a.MeetingID
        join EduComponents ec on ec.ComponentID = m.ComponentID
        WHERE u.UserID = @userID and a.Grade is not NULL and
            ec.ComponentID = @subjectID
        ORDER BY m.StartDate
    end
END
```

getAllUserGrades - zwraca wszystkie oceny użytkownika (Klient, Dyrektor) [WN]

```
ALTER   PROCEDURE [dbo].[getAllUserGrades]
    -- Add the parameters for the stored procedure here
    @userName nvarchar(MAX) = '',
    @userID int = -1

AS
BEGIN
    -- SET NOCOUNT ON added to prevent extra result sets from
    -- interfering with SELECT statements.
```

```
    SET NOCOUNT ON;

    IF @userID = -1 AND dbo.studentNameToUserIdNum(@userName) > 1
    BEGIN
        throw 51000, 'Wielu uczniów o tym samym imieniu i nazwisku', 1;
    END

    IF @userID = -1
    begin
        SELECT ec.Name as Subject, STRING_AGG(CAST(a.GRADE AS nvarchar), '; ') AS
GradeList, CAST(avg(a.Grade)as decimal(10,2)) as Average
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on u.UserID = s.UserID
        join Meetings m on m.MeetingID = a.MeetingID
        join EduComponents ec on ec.ComponentID = m.ComponentID
        WHERE (CONCAT(u.FirstName, ' ', u.LastName) = @userName) and a.Grade is
not NULL
        GROUP BY ec.Name
    end
    else
    begin
        SELECT ec.Name as Subject, STRING_AGG(CAST(a.GRADE AS nvarchar), '; ') AS
GradeList, CAST(avg(a.Grade)as decimal(10,2)) as Average
        FROM Attendance a join Subscriptions s on s.SubID = a.SubID
        join Users u on u.UserID = s.UserID
        join Meetings m on m.MeetingID = a.MeetingID
        join EduComponents ec on ec.ComponentID = m.ComponentID
        WHERE u.UserID = @userID and a.Grade is not NULL
        GROUP BY ec.Name
    end
END
```

AddToBasket - dodaje produkt do koszyka danego użytkownika (z OnlyAdvance ustawionym na 0). (Klient) [KN]

```
ALTER PROCEDURE [dbo].[AddToBasket]
    @UserID INT,
    @ProductID INT
AS
BEGIN
    exec removeUnpaidSubscriptions
    IF NOT EXISTS (SELECT 1 FROM Users WHERE UserID = @UserID)
    BEGIN
        RAISERROR ('Użytkownik nie istnieje', 16, 1);
        RETURN;
    END;

    if not exists (SELECT 1 from UserToRole where UserID=@UserID and RoleID=1)
    begin
        RAISERROR ('Użytkownik nie jest klientem', 16, 1);
        RETURN;
```

```
    END;


    IF NOT EXISTS (SELECT 1 FROM Products WHERE ProductID = @ProductID)
    BEGIN
        RAISERROR ('Produkt nie istnieje', 16, 1);
        RETURN;
    END;

    if (select isactive from products where ProductID=@ProductID)=0
    BEGIN
        RAISERROR ('Produkt jest nieaktywny', 16, 1);
        RETURN;
    END;

    if dbo.FreeSeats(@ProductID)=0
    BEGIN
        RAISERROR ('Brak miejsc', 16, 1);
        RETURN;
    END;

    if (select IsForSale from Products p join ProductTypes pt
    on pt.ProductTypeID=p.ProductTypeID
    where ProductID=@ProductID)=0
    BEGIN
        RAISERROR ('Produkt nie jest na sprzedaż', 16, 1);
        RETURN;
    END;

    IF EXISTS (SELECT 1 FROM Basket WHERE UserID = @UserID AND ProductID =
@ProductID)
    BEGIN
        RAISERROR ('Produkt już znajduje się w koszyku', 16, 1);
        RETURN;
    END;

    INSERT INTO Basket (UserID, ProductID,OnlyAdvance)
    VALUES (@UserID, @ProductID,0);

END;
```

PayOnlyAdvance - ustawia OnlyAdvance na 1, jeżeli jest możliwość zapłaty samej zaliczki dla podanego produktu. (Klient) [KN]

```
create procedure PayOnlyAdvance
    @UserID int,
    @ProdID int
as
begin
if not exists (select 1 from Basket
```

```
where @UserID=UserID
and @ProdID=ProductID)
begin
    raiserror ('Podany użytkownik nie posiada tego produktu w koszyku',16,1)
    return;
end;
if (select entryfee from Products p
where p.ProductID=@ProdID) is null
begin
    raiserror ('Płatność ratalna nie jest możliwa',16,1)
    return;
end;
update Basket
set OnlyAdvance = 1
where ProductID=@ProdID
and UserID=@UserID;
end;
```

PayFullPrice - zmienia OnlyAdvance na 0 (Klient)

```
create procedure PayFullPrice
    @UserID int,
    @ProdID int
as
begin
if not exists (select 1 from Basket
where @UserID=UserID
and @ProdID=ProductID)
begin
    raiserror ('Podany użytkownik nie posiada tego produktu w koszyku',16,1)
    return;
end;

update Basket
set OnlyAdvance = 0
where ProductID=@ProdID
and UserID=@UserID;
end;
```

DeleteFromBasket - usuwa wybrany produkt z koszyka użytkownika (Klient) [KN]

```
create procedure DeleteFromBasket
    @UserID int,
    @ProdID int
as
begin
if not exists (select 1 from Basket
where @UserID=UserID
and @ProdID=ProductID)
```

```
begin
    raiserror ('Podany użytkownik nie posiada tego produktu w koszyku',16,1)
    return;
end;

delete from Basket
where ProductID=@ProdID
and UserID=@UserID;
end;
```

BuyNow - przenosi produkty danego użytkownika z koszyka do subskrypcji z AccessAllowed ustawionym na 0 (wyjątek - darmowe webinary - AccessAllowed jest wtedy ustawiony na 1). (Klient) [KN]

```
ALTER PROCEDURE [dbo].[BuyNow]
    @UserID INT
AS
BEGIN
EXEC removeUnpaidSubscriptions;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (
            SELECT 1 FROM Basket WHERE UserID = @UserID
        )
        BEGIN
            THROW 50000, 'Podany użytkownik nie posiada produktów w koszyku', 1;
        END;
        IF EXISTS (
            SELECT 1
            FROM Subscriptions s
            JOIN Basket b ON b.UserID = s.UserID
            WHERE s.UserID = @UserID AND s.ProductID = b.ProductID
        )
        BEGIN
            THROW 50000, 'Podany użytkownik posiada już subskrypcję na produkt
...', 1;
        END;

        IF EXISTS (
            SELECT ProductID
            FROM Basket b
            WHERE UserID = @UserID AND dbo.FreeSeats(ProductID) = 0
        )
        BEGIN
            THROW 50000, 'Brak miejsc na produkt ...', 1;
        END;

        INSERT INTO Subscriptions (UserID, ProductID, AccessAllowed,
PaymentDeadline)
        SELECT DISTINCT UserID, p.ProductID, 0, DATEADD(MINUTE, 15, GETDATE())
        FROM Basket b
        JOIN Products p ON p.ProductID = b.ProductID
```

```
        WHERE UserID = @UserID AND p.FullPrice IS NOT NULL;

        DECLARE @Subscriptions TABLE (SubID INT, ProductID INT);
        INSERT INTO @Subscriptions (SubID, ProductID)
        SELECT DISTINCT s.SubID, s.ProductID
        FROM Subscriptions s
        JOIN Basket b ON b.ProductID = s.ProductID
        WHERE s.UserID = @UserID AND b.UserID = @UserID;

        IF EXISTS (SELECT 1 FROM @Subscriptions)
        BEGIN

            DECLARE @MainPaymentLink NVARCHAR(64);
            SET @MainPaymentLink = CONCAT('https://payment.', NEWID(), '.pl');

            INSERT INTO Payments (IsPaid, Link)
            VALUES (0, @MainPaymentLink);

            DECLARE @MainPaymentID INT = SCOPE_IDENTITY();

            INSERT INTO PaymentDetails (PaymentID, SubID, Value)
            SELECT DISTINCT
                @MainPaymentID,
                s.SubID,
                CASE
                    WHEN b.OnlyAdvance = 0 THEN p.FullPrice
                    ELSE p.EntryFee
                END AS Value
            FROM @Subscriptions s
            JOIN Products p ON s.ProductID = p.ProductID
            JOIN Basket b ON b.ProductID = p.ProductID AND b.UserID = @UserID
            WHERE p.SuperID IS NULL;
        END;

        INSERT INTO Subscriptions (UserID, ProductID, AccessAllowed)
        SELECT DISTINCT UserID, p.ProductID, 1
        FROM Basket b
        JOIN Products p ON p.ProductID = b.ProductID AND b.OnlyAdvance = 0
        WHERE UserID = @UserID AND p.FullPrice IS NULL;

        DELETE FROM Basket WHERE UserID = @UserID;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF XACT_STATE() <> 0
        BEGIN
            ROLLBACK TRANSACTION;
        END;
        THROW;
    END CATCH;
END;
```

removeUnpaidSubscriptions - usuwa subskrypcje, które nie mają przyznanego dostępu, a upłynął termin płatności (usuwa również dane tych płatności). [KN]

```
ALTER procedure [dbo].[removeUnpaidSubscriptions]
as
begin
    delete from PaymentDetails
    where SubID in (select SubID
    from Subscriptions
    where AccessAllowed=0
    and GETDATE()>PaymentDeadline)

    delete from Subscriptions
    where AccessAllowed=0
    and GETDATE()>PaymentDeadline

    delete from Payments
    where not exists
    (select 1 from PaymentDetails pd
    where pd.PaymentID=paymentid)
end;
```

setDeferredPaymentConsent - procedura dla dyrektora, do ustawiania zgody na płatność odroczoną dla podanej subskrypcji i wyznaczenia daty wymaganej płatności. (Dyrektor) [KN]

```
CREATE PROCEDURE setDeferredPaymentConsent
    @SubID INT,
    @PaymentDate DATETIME
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        UPDATE Subscriptions
        SET AccessAllowed = 1,
            PaymentDeadline = @PaymentDate
        WHERE SubID = @SubID;

        IF @@ROWCOUNT = 0
        BEGIN
            THROW 50001, 'Subskrypcja o podanym SubID nie została znaleziona.', 1;
        END

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

registerClient - procedura rejestrująca nowego użytkownika i przyznająca mu rolę klienta (Użytkownik niezalogowany - proces tworzenia konta klienta) [KN]

```
CREATE PROCEDURE registerClient
    @FirstName NVARCHAR(20),
    @LastName NVARCHAR(20),
    @Email NVARCHAR(40),
    @Address NVARCHAR(100),
    @Password NVARCHAR(20)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        INSERT INTO Users (FirstName, LastName, Email, Address, Password)
        VALUES (@FirstName, @LastName, @Email, @Address, @Password);
        DECLARE @UserID INT = SCOPE_IDENTITY();

        INSERT INTO UserToRole (UserID, RoleID)
        VALUES (@UserID, (SELECT RoleID FROM Roles WHERE RoleName = 'Klient'));

        COMMIT TRANSACTION;
    END TRY

    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH;
END;
```

registerUser - rejestracja użytkownika bez podania roli i adresu (adres jest wymagany tylko dla klientów) (Administrator) [KN]

```
CREATE PROCEDURE registerUser
    @FirstName NVARCHAR(20),
    @LastName NVARCHAR(20),
    @Email NVARCHAR(40),
    @Password NVARCHAR(20)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        INSERT INTO Users (FirstName, LastName, Email, Password)
        VALUES (@FirstName, @LastName, @Email, @Password);

        COMMIT TRANSACTION;
    END TRY
```

```
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH;
END;
```

AssignRole - przypisuje rolę podanemu użytkownikowi. Rolę klient można przypisać tylko, jeżeli użytkownik podał adres. (Administrator) [KN]

```
ALTER PROCEDURE AssignRole
    @UserID INT,
    @RoleID INT
AS
BEGIN
    BEGIN TRY

        IF EXISTS (SELECT 1 FROM Users WHERE UserID = @UserID)
        BEGIN
            if (@RoleID=1 and (select Address from Users where UserID=@UserID) is
null)
            begin
                throw 52000, 'Najpierw należy uzupełnić adres dla podanego
użytkownika.', 1;
            end
            else
            IF EXISTS (SELECT 1 FROM Roles WHERE RoleID = @RoleID)
            BEGIN
                INSERT INTO UserToRole (UserID, RoleID)
                VALUES (@UserID, @RoleID);
            END
            ELSE
            BEGIN
                THROW 52000, 'Nie ma roli o podanym ID', 1;
            END
        END
        ELSE
        BEGIN
            THROW 52000, 'Użytkownik o podanym ID nie istnieje',1;
        END
    END TRY
    BEGIN CATCH
        DECLARE @msg nvarchar(2048)= ERROR_MESSAGE();
        THROW 52000, @msg, 1;
    END CATCH
END;
```

addAddress - aktualizuje adres podanego użytkownika. (wszystkie role) [KN]

```sql
alter PROCEDURE addAddress
    @UserID INT,
    @Address NVARCHAR(100)
AS
BEGIN
    BEGIN TRY
        UPDATE Users
        SET Address = @Address
        WHERE UserID = @UserID;

        IF @@ROWCOUNT = 0
        BEGIN
            THROW 50000, 'Nie znaleziono użytkownika o podanym ID', 1;
        END
    END TRY
    BEGIN CATCH
        DECLARE @msg nvarchar(2048)= ERROR_MESSAGE();
        THROW 50000, @msg, 1;
    END CATCH
END;
```

RemoveUsersRole - odbiera użytkownikowi rolę. (Administrator) [KN]

```sql
CREATE PROCEDURE [dbo].[RemoveUsersRole]
    @UserID INT,
    @RoleID INT
AS
BEGIN

    BEGIN TRY
        IF NOT EXISTS (SELECT 1 FROM Users WHERE UserID = @UserID)
        BEGIN;
            THROW 52000, N'Użytkownik o podanym UserID nie istnieje.', 1;
        END

        IF NOT EXISTS (SELECT 1 FROM Roles WHERE RoleID = @RoleID)
        BEGIN;
            THROW 52000, N'Rola o podanym RoleID nie istnieje.', 1
        END

        IF NOT EXISTS (SELECT 1 FROM UserToRole WHERE UserID = @UserID AND RoleID
= @RoleID)
        BEGIN;
            THROW 52000, N'Rola nie jest przypisana do tego użytkownika.', 1;
        END;

        DELETE FROM UserToRole
        WHERE UserID = @UserID AND RoleID = @RoleID

    END TRY
    BEGIN CATCH
```

```
        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
        THROW 52000, @ErrorMessage, 1
    END CATCH

END
```

ReceivedDiploma - procedura do zaznaczenie przez dyrektora kto i za co otrzymał dyplom. (Dyrektor) [KN]

```
alter procedure ReceivedDiploma
@SubID int
as
begin
begin try
    if exists (select 1 from Subscriptions where SubID=@SubID and
dbo.GetProductTypeName(ProductID) in ('kurs', 'studia') and IsPassed=1)
    begin
        begin
        update Subscriptions
        set ReceivedDiploma=1
        where SubID=@SubID
        end
    end
    else
    throw 50000,'Nie można wystawić dyplomu za podaną subskrypcję lub podana
subskrypcja nie istnieje',1;
end try
begin catch
    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
    THROW 50000, @ErrorMessage, 1
END CATCH
end
```

TeacherSchedule - plan nadchodzących spotkań nauczyciela (Nauczyciel, Dyrektor) [WN]

```
CREATE or ALTER PROCEDURE TeacherSchedule
    @teacherID int,
    @todayDate date
AS
BEGIN
    SET NOCOUNT ON;
    SELECT *
    FROM Meetings m
    WHERE m.TeacherID = @teacherID and m.StartDate > @todayDate
    order by m.StartDate
END
GO
```

TranslatorSchedule - plan nadchodzących spotkań dla tłumacza (Tłumacz, Dyrektor) [WN]

```sql
ALTER   PROCEDURE [dbo].[TranslatorSchedule]
    -- Add the parameters for the stored procedure here
    @translatorID int,
    @todayDate date
AS
BEGIN
    -- SET NOCOUNT ON added to prevent extra result sets from
    -- interfering with SELECT statements.
    SET NOCOUNT ON;

    -- Insert statements for procedure here
    SELECT *
    FROM Meetings m
    JOIN Translations t on t.MeetingID = m.MeetingID
    WHERE t.TranslatorID = @translatorID and m.StartDate > @todayDate
END
```

ProductSiege - pokazuje obleganie produktów [WN]

```sql
CREATE or Alter PROCEDURE ProductSiege

    @todayDate date
AS
BEGIN

    SET NOCOUNT ON;

    SELECT p.ProductID, p.ProductName,
        (select pt.ProductTypeName
        from ProductTypes pt
        where p.ProductTypeID = pt.ProductTypeID) as Type,
        dbo.FindStartDate(p.ProductID) as [Start Date],
        dbo.FreeSeats(p.ProductID) as [Free Seats],
        p.MaxSeats,
        (select count(*)
        from Subscriptions s
        where s.ProductID = p.ProductID
        ) as [People Enrolled]
    FROM Products p
    WHERE dbo.FindStartDate(p.ProductID) > @todayDate
    ORDER BY [Start Date]
END
GO
```

ProductPassUpdate - aktualizacja statusu zaliczenia produktu na podstawie obecności oraz ocen z egznaminów (Administrator, Nauczyciel) [WN]

```sql
ALTER PROCEDURE [dbo].[ProductPassUpdate]
    @userID INT,
    @prodID INT
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT EXISTS (
        SELECT 1
        FROM EduComponents ec
        WHERE ec.ProductID = @prodID
        AND (dbo.areExamsPassed(@userID, ec.ComponentID) = 0
             OR dbo.calculateFrequency(@userID, ec.ComponentID) = 0)
    )
    and EXISTS (
        SELECT s.SubID
        FROM Subscriptions s
        WHERE s.ProductID = @prodID AND s.UserID = @userID AND s.isPassed = 0
    )
    BEGIN

        UPDATE Subscriptions
        SET isPassed = 1
        WHERE ProductID = @prodID AND UserID = @userID;
    END
END;
```

[KN] - Karolina Nitsch [WN] - Witold Nieć